

MOVE / MATCH · VOICE CONTROL

语音控制开发流程与接入指南

从浏览器语音识别、基础命令解析，到后续 VLM 语义定位与视频执行

阶段 1 已实现

独立运行

Swagger 已登记

文档目标

说明每个开发步骤、对应文件、验收方法和未来扩展边界。文档不展开具体代码，只说明需要在什么文件里实现什么能力。

当前日期

2026-07-20

当前阶段的目标与边界

先把语音链路做通，再接入视频和 VLM。

本阶段已经实现什么

用户在 AI 教学页点击一次语音按钮后，浏览器尝试持续监听中文；识别到完整文本后调用 NestJS 指令解析接口；后端返回结构化命令；前端显示识别结果。浏览器不支持或麦克风不可用时，可使用文字输入完成同一条链路测试。

整体数据流

麦克风 → 浏览器 ASR → NestJS API → 命令解析 → 界面确认

阶段边界

- 只识别和解析简单命令，不直接调用视频播放器、录制器或 VLM。
- 复杂动作语义，例如“手腕绕圈的位置”，明确返回暂不支持，不猜测执行。
- 后端仅允许固定命令集合，避免模型产生不可控动作。
- 所有 REST API 继续进入 Swagger，便于后续成员联调。

为什么现在不使用 Agent：基础命令是有限集合，确定性规则更快、更稳定、更容易测试。Agent 或 LLM 规划应在 VLM 与视频工具具备后再增加。

验收标准

检查项	通过条件
浏览器输入	Chrome 或 Edge 能请求麦克风并产生中文转写。
无麦克风测试	文字测试输入可以调用完全相同的后端解析 API。
后端解析	支持的指令返回 intent、参数、置信度和确认文案。
未知命令	返回 accepted=false，不执行任何动作。
工程质量	前后端 lint、build 和后端单元测试全部通过。

前端文件与作用

文件	需要完成的功能	当前状态
ftnd/src/features/voice-control/components/VoiceControlPanel.tsx	编写语音按钮、持续监听状态、识别结果、错误提示和文字测试对话框。	已实现
ftnd/src/features/voice-control/hooks/useBrowserSpeechRecognition.ts	封装浏览器 SpeechRecognition、中文设置、自动续听、结果回调和权限错误。	已实现
ftnd/src/features/voice-control/api.ts	编写调用后端 interpret 接口的请求函数。	已实现
ftnd/src/features/voice-control/types.ts	定义命令意图、参数、返回结果和状态类型。	已实现
ftnd/src/features/voice-control/index.ts	统一导出组件、API 和公开类型。	已实现
ftnd/src/features/ai-teaching/AITeachingPage.tsx	保留 VoiceControlPanel 接入点；当前不接视频执行回调。	已接入

开发步骤

步骤	动作	文件
1	建立浏览器语音 Hook，设置 zh-CN、连续监听和最终文本回调。	hooks/useBrowserSpeechRecognition.ts
2	把最终转写文本发送给后端，而不是在组件中直接判断命令。	api.ts
3	在 Material UI 对话框中展示监听、解析、成功和失败状态。	components/VoiceControlPanel.tsx
4	增加文字测试输入，保证没有麦克风时仍可独立验收后端。	components/VoiceControlPanel.tsx
5	通过可选回调暴露识别结果，为未来视频执行器保留入口。	components/VoiceControlPanel.tsx
6	从 index.ts 导出公共能力，禁止其他模块跨目录引用内部 Hook。	index.ts

浏览器限制：Web Speech API

的可用性取决于浏览器和网络服务。本阶段用它快速验证产品链路；后续可在保持同一 API 契约的情况下替换为本地 sherpa-onnx。

后端文件与作用

文件	需要完成的功能	当前状态
bknd/src/voice-control/voice-control.controller.ts	提供 POST /api/voice/commands/interpret，并维护 Swagger 装饰器。	已实现
bknd/src/voice-control/dto/interpret-voice-command.dto.ts	校验文本非空、字符串类型和最大 200 字符。	已实现
bknd/src/voice-control/voice-control.service.ts	清洗文本、匹配固定指令、提取秒数与倍速并返回安全结果。	已实现
bknd/src/voice-control/contracts/voice-command.types.ts	定义允许的 intent、参数和统一响应结构。	已实现
bknd/src/voice-control/voice-control.module.ts	注册 Controller 和 Service，并继续向 AI Teaching 导出服务。	已实现
bknd/src/voice-control/voice-control.service.spec.ts	覆盖常用命令、参数提取和未知命令。	已实现
bknd/src/voice-control/index.ts	统一导出 Module、Service 和公开契约。	已实现

接口契约

POST /api/voice/commands/interpret

输入：浏览器识别得到的 transcript。
输出：accepted、intent、confidence、parameters、responseText 和 executionStatus。
executionStatus 当前固定为 not-dispatched，表示只识别、不执行。

后端安全规则

- 只有 contracts 中声明的 intent 可以返回。
- 未知、歧义或越界倍速返回不接受，不自动寻找近似动作。
- “停止录制”等具体指令优先于“暂停”等短关键词，避免错误匹配。
- 未来加入 LLM 时，LLM 输出仍必须经过 DTO 与 allowlist 校验。

当前支持的简单指令

这些指令可以独立识别，但暂时不会控制视频。

用户说法示例	返回 intent	提取参数	当前确认结果
暂停 / 停一下 / 等一下	PAUSE	无	识别为暂停
继续 / 继续播放 / 接着	RESUME	无	识别为继续
慢一点 / 太快了 / 慢放	SLOW_DOWN	无	降低播放速度
快一点 / 太慢了 / 加速	SPEED_UP	无	提高播放速度
调到 0.5 倍	SET_PLAYBACK_RATE	playbackRate=0.5	设置指定倍速
倒回五秒 / 回退 3 秒	REWIND	seconds	倒回指定秒数
快进五秒 / 往后跳 3 秒	FAST_FORWARD	seconds	快进指定秒数
重新播放 / 从头开始	RESTART	无	重新开始
开始录制 / 开始录像	START_RECORDING	无	开始录制
停止录制 / 结束录像	STOP_RECORDING	无	停止录制

暂不支持的指令

示例	原因	后续依赖
倒回刚才手腕绕圈的位置	需要理解视频动作语义和时间位置。	VLM 动作定位 + 视频时间轴
把这个动作拆成三步	需要理解动作过程并生成教学步骤。	VLM 动作拆解
我的手哪里做错了	需要比较原视频和跟练画面。	双路画面 + VLM 纠错
继续副歌第一个动作	需要歌曲段落和动作片段标签。	数据时间轴 + 视频执行器

暂不支持的复杂命令必须返回 accepted=false。这样可以验证语音链路，又不会在 VLM 和视频能力不存在时给用户错误反馈。

启动、测试与验收步骤

以下步骤可由任何队友在新电脑上重复。

A. 启动环境

顺序	操作	验证
1	启动 PostgreSQL 容器。	数据库端口 5434 可连接。
2	在 bknd 执行 <code>npm ci</code> 和 <code>npm run start:dev</code> 。	<code>http://localhost:3001/api/health</code> 返回 200。
3	在 ftnd 执行 <code>npm ci</code> 和 <code>npm run dev</code> 。	<code>http://localhost:3000</code> 可以打开。
4	登录后进入 <code>/teaching</code> 。	页面下方显示“语音输入”按钮。

B. 测试后端

- 在 Swagger 打开 `POST /api/voice/commands/interpret`。
- 输入“倒回五秒”，确认返回 `REWIND` 和 `seconds=5`。
- 输入“调整到 0.5 倍”，确认返回 `SET_PLAYBACK_RATE`。
- 输入“拆解这个动作”，确认 `accepted=false`。

C. 测试前端

- 点击“语音输入”，允许麦克风权限并说出简单指令。
- 观察实时文本、解析状态和最终命令代码。
- 如果麦克风不可用，在文字测试中输入同样指令。
- 连续测试“停止录制”和“暂停”，确认不会混淆。

D. 自动检查

后端依次执行：`npm run lint`、`npm test -- --runInBand`、`npm run build`。
前端依次执行：`npm run lint`、`npm run build`。

本次实际验证结果

后端 11 个单元测试全部通过；前端和后端 lint、生产构建全部通过；Swagger 已出现语音接口；浏览器文字测试“倒回五秒”成功显示 `REWIND`。

从浏览器 ASR 升级到开源本地 ASR

建议目标: sherpa-onnx-node + 中文流式模型。

为什么保留当前实现

- 浏览器 ASR 能最快验证麦克风、界面、后端指令契约和错误流程。
- 命令解析位于 NestJS, 替换 ASR 时不需要重写命令规则。
- 文字测试可长期保留, 方便自动化和无麦克风环境调试。

升级时建议增加的文件

建议文件	需要编写的功能	是否现在需要
bknd/src/voice-control/providers/asr-provider.interface.ts	定义统一的音频转写 Provider 接口。	下一阶段
bknd/src/voice-control/providers/sherpa-onnx-provider.ts	加载本地模型并输出流式中文文本。	下一阶段
bknd/src/voice-control/voice-stream.gateway.ts	接收浏览器音频流; WebSocket 需单独补充接口说明。	按需
ftnd/src/features/voice-control/hooks/useAudioStream.ts	采集并发送浏览器 PCM 音频。	下一阶段
bknd/.env.example	增加 ASR_PROVIDER 和 SHERPA_MODEL_DIR 等变量名。	接入时

升级顺序

步骤	说明
1	先在独立脚本中验证 sherpa-onnx-node 能加载中文模型。
2	建立 AsrProvider 接口, 把模型实现封装在 Provider 中。
3	增加浏览器音频传输, 不修改 VoiceControlPanel 的命令结果结构。
4	用相同的十条命令执行准确率和延迟测试。
5	保留 Web Speech Provider 作为比赛现场备用方案。

模型文件管理: 模型通常较大, 不提交到 Git。通过下载脚本或共享存储分发, 模型目录写入后端环境变量, 并在 .gitignore 中忽略。

接入 VLM 与视频后的完整路线

简单命令层保持不变，在其后增加规划和工具执行。

未来完整链路



模块职责

模块	负责人	未来需要提供的能力
Voice Control	你	ASR、上下文、复杂指令规划、工具顺序、反馈。
VLM Core	VLM 同学	动作语义定位、动作对比、错误解释、三步拆解。
Video Stage	视频同学	seek、倍速、暂停、片段播放、当前时间和帧采集。
Data Pipeline	数据同学	动作时间轴、片段标签、分析结果和媒体 ID。

复杂指令示例的执行顺序

用户指令：倒回刚才手腕绕圈的位置，放慢到 0.5 倍，然后拆成三步讲给我。

顺序	工具动作	依赖模块
1	ASR 得到完整文字。	Voice Control
2	规划器拆成定位、跳转、倍速、拆解和播报五步。	Voice Control
3	根据当前时间查找之前的“手腕绕圈”片段。	VLM Core + Data
4	跳到片段起点并设置 0.5 倍速。	Video Stage
5	请求三步动作拆解并展示、播报。	VLM Core + Voice Control

建议增加的编排文件

文件	作用
bknd/src/voice-control/voice-instruction-planner.service.ts	把复杂文本转换为受限工具计划。
bknd/src/voice-control/voice-command-executor.service.ts	按顺序执行允许的工具并记录结果。
bknd/src/voice-control/contracts/voice-plan.types.ts	定义计划、步骤、参数和执行状态。
ftnd/src/features/voice-control/types.ts	扩展复杂计划进度与确认状态。

开发里程碑与交接清单

每个阶段都应独立可演示、可回退。

阶段	交付内容	完成标准
阶段 1: 基础语音	浏览器转写 + 简单命令 API + UI 确认。	已完成并通过测试。
阶段 2: 本地 ASR	sherpa Provider + 音频流 + 浏览器备用 Provider。	断网时仍可识别十条命令。
阶段 3: 视频执行	简单 intent 绑定到播放器和录制器。	暂停、倍速、回退、录制可语音执行。
阶段 4: VLM 检索	动作时间轴、语义定位和置信度。	能定位“刚才手腕绕圈”。
阶段 5: 复杂编排	规划器 + allowlist 工具执行 + 失败回退。	能完成复合指令并逐步反馈。
阶段 6: 比赛加固	离线备用、日志、延迟、异常和演示脚本。	现场网络变化不影响核心演示。

提交前检查

- 新增 REST API 已显示在 Swagger，并包含请求 DTO 和响应示例。
- 前端没有保存 API Key；模型密钥和模型路径只在后端环境变量中。
- 复杂指令在依赖未接入时不会被错误识别为简单命令。
- 其他模块只从 voice-control/index.ts 引用公开能力。
- 模型文件、真实录音和测试密钥没有进入 Git。
- README 只维护环境、安装和运行说明；开发流程使用本 PDF。

官方参考

Web Speech API: developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API

sherpa-onnx: github.com/k2-fsa/sherpa-onnx

sherpa-onnx Node API: k2-fsa.github.io/sherpa/onnx/javascript-api

项目 Swagger: <http://localhost:3001/api/docs>

下一步建议

先让你在 Chrome 或 Edge 中实测十条简单命令。确认麦克风、中文转写和命令结果稳定后，下一次开发直接接 Video Stage 的安全执行器；本地 ASR 和 VLM 可以并行推进。